T10

Group 12
EShop Case Study

2026 AI-First
CS423/CSC13003

FIT @ HCMUS - Department of Software Engineering

Mutation Testing & Test Effectiveness

Can our tests detect a wrong result?
StrykerJS, reviewed assertions and real EShop mutants

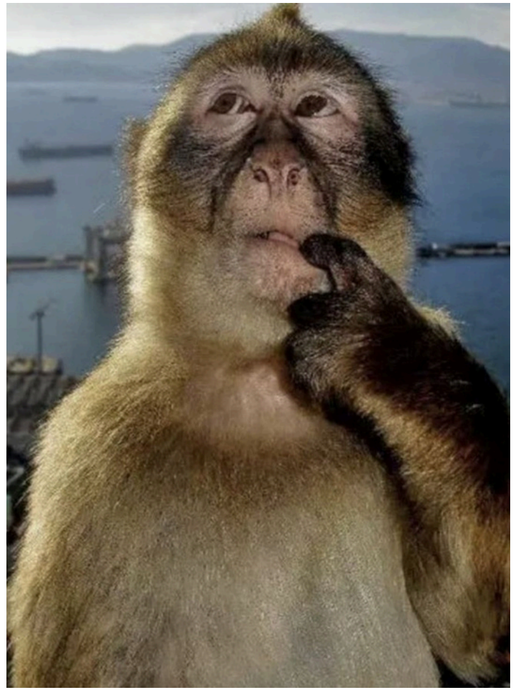
23120255 - Lê Tấn Hiệp
23120262 - Tống Dương Thái Hòa
23120264 - Nguyễn Phúc Hoàng

Passing tests and test effectiveness

All tests pass. What have we actually checked?

Coverage records which code ran.
Assertions decide whether the result is acceptable.

A test that checks only HTTP 200 may miss an incorrect cart, discount or account state.



By the end of this session

1. Read mutant outcomes and the score denominator
2. Explain survivors using inputs, observations and the specification
3. Write and review assertions, then verify a target kill

Mutation testing workflow

A mutant is a small code change used to challenge the tests.

01 Original code	02 Mutate	03 Retest	04 Inspect
Run the tests first	Change one expression	Use the same tests	Read state and test

Teaching example	Original	Mutant
Rule	amount >= 100	amount > 100
Input: 101	true	true
Input: 100	true	false



Input 100 exposes the boundary.
The test must assert the returned value.

Mutant states and mutation score

State	Meaning in this report
Killed (K)	At least one test failed
Survived (S)	Selected tests passed
NoCoverage (N)	No test covered the change
Timeout (T)	Execution exceeded the limit
RuntimeError	Runner or runtime could not finish normally

RuntimeError and CompileError are outside V.
Ignored and Pending also stay outside the score.

Stryker score

V = K + T + S + N

Score = (K + T) / V

Baseline: 252 / 373 = 67.56%

We report K and T separately. K / V is an additional assertion-kill metric.

StrykerJS and the AI Agent

StrykerJS: mutation execution

Fits the JavaScript / Node.js backend
Runs the selected Jest tests
Exports HTML and JSON evidence

AI Agent: assertion candidates

Inputs: rule, mutant, tests, fixtures
Output: candidate + expected behavior
Human checks the oracle and setup

Reusable review and verification loop

1. Explain the survivor and propose an assertion
2. Review against the specification and run on original code
3. Rerun the same mutant and inspect the detecting test



Keep the production source and configuration fixed. Save prompt, edits and reports.

EShop experiment design

Domain	Baseline tests	Behavior under test
Cart & Orders	36	Cart state, checkout and orders
Coupons	26	Discount rules and usage limits
Authentication & Users	26	Login, reset and profile access

Shared production baseline: 806ba19

395 matching mutant signatures across all three joint runs
Original-code tests: baseline 88/88, manual-analysis 118/118, AI 118/118
Same source, mutation scope, timeout policy and isolated data setup

Inputs are handed-over collections. Equal test counts do not imply independent generation.

Case 1: the empty-cart oracle

Rule: a newly created cart contains no items.

Original

```
userCarts[userId] = [];
```

Mutant: ArrayDeclaration

```
userCarts[userId] =
["Stryker was here"];
```

The missing observation

Weak check: expect(empty.status).toBe(200)
Stronger check: expect(empty.body).toEqual([])

A successful request can still return incorrect data. Inspect the response body.

Case 2: coupon usage-limit data

Original expression	Mutant replacement
max_uses_per_user || 1	max_uses_per_user && 1

Admin input	Original stores	Mutant stores	Test distinguishes?
1	1	1	No
3	3	1	Yes

MF-01 reads the created coupon back:
expect(listed.body[0].max_uses_per_user).toBe(3)

Domain reports: two related survivors became Killed. More cases added no further kills.

Case 3: a profile update keeps the role

Rule: a profile update without a role field preserves the stored role.

Original condition	Mutant: force the role-update branch
if (role) { … }	if (true) { … }

AM-01 checks database state and API output

```
expect((await testApi.userByEmail(ALICE.email)).role)
  .toBe('user');
expect((await me(token)).body.role).toBe('user');
```

Auth domain report: mutant #132 changed from Survived to Killed.

Joint results on 395 matching mutants

Collection	Tests	Killed	Timeout	Survived	NoCov.	Error	Score
Baseline	88	191	61	87	34	22	67.56%
Manual-analysis	118	238	64	43	30	20	80.53%
Reviewed AI	118	211	72	62	30	20	75.47%

Score = (Killed + Timeout) / valid mutants. Error = RuntimeError.
Valid denominators: 373, 375, 375. CompileError / Ignored / Pending: 0.

New Killed from baseline Survived or NoCoverage

48 manual-analysis

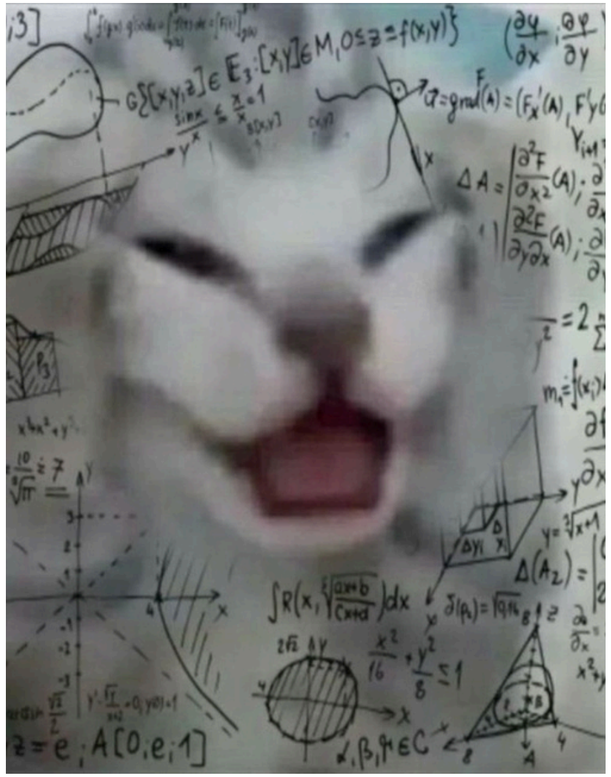
28 reviewed AI

K / V: 51.21% baseline, 63.47% manual-analysis, 56.27% reviewed AI

Equivalent mutants and limits

Why can a mutant survive?

Missing input or check: add a spec-based test.
Equivalent: same observations for all valid inputs.
Auth candidate: counts 0, 2, 4 mask >= 3 vs > 3.



What AI can contribute

Propose counterexamples and equivalence arguments.
Review inputs, side effects and assumptions.
Keep unproven cases labeled equivalent candidates.

These runs compare collections, not model ability. Coupons used Codex on both sides. Timeout, SQLite errors, oracle conventions and machine load limit the conclusion.

Live demo: verify the coupon mutant

StrykerJS on the real EShop backend

1. Run the passing baseline and open the mutation report
2. Inspect line 474: input 1 leaves the mutant Survived
3. Add input 3 and verify the stored value on original code

AI candidate, review and execution

4. Inspect the recorded prompt and Codex candidate
5. Check the rule and fixture, then run the AI test collection
6. Match the same mutant and inspect its detecting test



Activity: Kill the Mutant

Teams of 3-4. Write one candidate assertion per case.
Use the supplied worksheet and fixtures. Jest or clear pseudocode is acceptable.

Case	Mutant change	Rule to protect
K1	New cart contains an extra item	A new cart is empty
K2	Remove the push operation	Added items persist
K3	Reset cart on every read	Reading keeps cart data
K4	Remove the order-query parameter	History includes the user's order
K5	Save an empty cancellation status	Cancellation is persisted

Use worksheet context: empty, cart, reread, history and canceled. No AI during scoring.

Activity steps and scoring

Time	Team action
00-02 min	Read the five diffs, rules and fixture data
02-10 min	Write five candidate assertions
10-14 min	Swap, review and revise with a peer team
14-18 min	Run on original and target mutants, then tally
18-20 min	Explain one design and submit the response sheet

One point per verified target kill. Maximum: 5 points.

Original code must pass. The candidate must fail because of its target mutant.
Syntax errors, setup failures and Timeout receive no kill point.

Minute paper: Which observation mattered? What would you check next?

Takeaways and Q&A

Three things to remember

1. Reach the change with useful input.
2. Check behavior against the rule.
3. Verify the original pass and target kill.



Further reading

Stryker: states, metrics and equivalent mutants
Jia & Harman: mutation-testing survey (2011)
Group 12: report, guide and activity worksheet

ANY QUESTION BRUH?

Which survivor would you investigate next, and what evidence would convince you?